FIELD PROJECT / FIELD VISIT REPORT

DATA ANALYTICS OF SUPERMARKET SALES AND PRODUCT TRENDS USING PYTHON

Based on a Field Visit to Choice Super Market, Virar (East), Vasai-Virar

Subject: Data Analytics with Python Lab

Field Project / Field Visit

Student Name: [MY NAME]

Roll No.: [MY ROLL NO.]

Class: [CLASS / DIVISION]

Academic Year: 2026–27

College: [COLLEGE NAME]

Teacher / Faculty: [TEACHER NAME]

CERTIFICATE

[COLLEGE NAME]

This is to certify that Mr. / Ms. _____ , Roll No. _____ of Class _____ has satisfactorily completed the Field Project / Field Visit Report titled

“DATA ANALYTICS OF SUPERMARKET SALES AND PRODUCT TRENDS USING PYTHON”

in the subject Data Analytics with Python Lab during the academic year 2026–27. The work presented in this report is carried out by the student under the guidance of the faculty and includes a field visit, data cleaning, processing, analysis and visualisation using Python.

Date: ____ / ____ / 2026

_____	_____	_____
Faculty Signature	HOD Signature	Principal / I/C Principal

College Stamp

INDEX

Section	Topic	Type	Page No.
1	Introduction	Written	4
2	Source of Dataset	Written + Printout	6
3	Purpose and Scope	Written	8
4	Field Visit Description	Written	8
5	Geotagged Photographs (Figures 1–3)	Photographs	10
6	Objectives of the Field Project	Written	13
7	Technologies and Tools Used	Written	13
8	Dataset Description	Printout	14
9	Data Loading	Printout	16
10	Data Cleaning	Printout	19
11	Data Processing / Feature Engineering	Printout	22
12	Exploratory Data Analysis	Printout	24
13	Data Summarization	Printout	29
14	Data Visualization (Figures 4–19)	Printout	32
15	Analysis and Interpretation	Written	41
16	Key Findings	Written	42
17	Conclusion	Written	44
18	Limitations	Written	44
19	Future Scope	Written	44
20	References	Written	44

Note: Page numbers are for this typed file. Adjust them to match your final paper pages after handwriting.

1. INTRODUCTION

1.1 Introduction to Data Analytics

Data analytics is the process of collecting, cleaning, organising and examining data to find useful patterns and support decision-making. It usually follows a series of steps: data collection, data cleaning, data processing, analysis, visualisation and interpretation. Today, organisations of every size use data analytics to understand their customers, improve their services and reduce their costs. Python is one of the most popular languages for this work because it is simple to learn and has powerful libraries for handling and visualising data.

1.2 Introduction to Supermarket Data Analytics

A supermarket sells many types of products such as groceries, beverages, snacks, personal care items and household cleaning products. Every time a customer buys something, a record is created at the billing counter that shows what was sold, in what quantity, at what price, on which date and how the payment was made. When such records are collected over time, they form a dataset. Supermarket data analytics means studying these records to understand how the business is performing.

1.3 Importance of Data Analytics in Retail and Supermarkets

- It shows which products and categories bring the most sales.
- It helps the owner decide what quantity of each product should be stocked.
- It helps in planning prices, offers and discounts.
- It helps to identify slow-moving products that occupy shelf space.
- It shows sales patterns over days and months, which helps in planning for busy periods.
- It reduces guesswork and supports decisions with actual numbers.

1.4 How Data Can Help Understand Customer Purchases

Purchase records show what customers buy, how many units they buy at a time, how often certain products appear in bills and how they prefer to pay. Grouping the data by customer type, payment method or date reveals buying habits. For example, bulk buyers usually purchase larger quantities than occasional customers, and sales on weekends may differ from sales on weekdays.

1.5 Importance of Sales, Product, Category and Pricing Analysis

- Sales analysis tells us the total revenue and how it changes with time.
- Product analysis identifies best-selling and lowest-selling items.
- Category analysis compares groups of products such as Beverages and Home Care.
- Pricing analysis shows the range of prices and how price is related to quantity and total sales.
- Discount analysis shows how much discount was given and whether it is related to higher sales.

1.6 Brief Introduction to the Selected Field Visit

For this field project, a supermarket in Virar (East), Vasai-Virar, Maharashtra was visited on 29/09/2026 in the evening. The name on the shop signboard, as seen in Figure 1, is Choice

Super Market. Geotagged photographs were taken inside and outside the shop to record the environment in which retail data is generated. These photographs are given in Section 5.

1.7 Scope of the Project

The project covers a visit to a supermarket, followed by the analysis of a supermarket sales dataset using Python. The analysis includes data loading, cleaning, processing, exploratory analysis, summarisation and visualisation of sales, category, product, discount, payment and customer-type data. The conclusions are based only on the dataset used in this project.

2. SOURCE OF DATASET

The supermarket's actual billing records are confidential and were not available for this project. Therefore, a sample sales dataset was created using Python (NumPy and pandas) for analytical demonstration. The dataset was generated on the computer using a fixed random seed (42), so it can be reproduced exactly by running the code given at the end of this section.

The dataset used for analytical demonstration is a simulated/sample dataset and is not claimed to represent the actual sales records of the visited supermarket.

2.1 Details of the Dataset

- Type of data: Simulated / sample data (not real field data).
- How it was obtained: Generated by the student using Python code (given below); file name: supermarket_sales_raw.csv.
- What it represents: Bill-line sales records of a typical supermarket for the period 01-07-2026 to 28-09-2026.
- Number of records and columns (raw file): 1,215 rows and 14 columns.
- Products: 42 products in 6 categories (Staples & Grocery, Beverages, Snacks & Confectionery, Personal Care, Home Care, Dairy & Bakery). The categories were chosen to match the kinds of products seen in the field-visit photographs.
- Brands, prices and quantities are assumed values; brand names are fictional and prices do not represent the actual prices of the visited shop.
- The raw file deliberately contains typical data problems (missing values, duplicate rows, inconsistent category names and invalid entries) so that data cleaning can be demonstrated.

2.2 Important Variables

Product_ID, Product_Name, Category, Sub_Category, Brand, Unit_Price, Quantity, Discount_Percent, Date, Payment_Method, Customer_Type and Stock_Available. Total Sales, Discount Amount and Final Sales are calculated later in Section 11.

2.3 Code Used to Create the Sample Dataset

Command / Code

```
import numpy as np
import pandas as pd

np.random.seed(42)

products = [
    ("P101", "Basmati Rice 5 kg", "Staples & Grocery", "Rice", "GoldenGrain", 620),
    ("P102", "Wheat Flour 5 kg", "Staples & Grocery", "Flour", "HarvestPure", 260),
    ("P103", "Toor Dal 1 kg", "Staples & Grocery", "Pulses", "GoldenGrain", 165),
    ("P104", "Sunflower Oil 1 L", "Staples & Grocery", "Edible Oil", "SunDrop", 145),
    ("P105", "Sugar 1 kg", "Staples & Grocery", "Sugar", "SweetLeaf", 48),
    ("P106", "Iodised Salt 1 kg", "Staples & Grocery", "Salt", "PureCrystal", 26),
    ("P107", "Tea Powder 500 g", "Staples & Grocery", "Tea", "MorningBrew", 240),
    ("P201", "Orange Juice 1 L", "Beverages", "Juices", "FruitFresh", 110),
    ("P202", "Mixed Fruit Juice 1 L", "Beverages", "Juices", "FruitFresh", 115),
    ("P203", "Cola 750 ml", "Beverages", "Soft Drinks", "FizzUp", 40),
    ("P204", "Lemon Soda 750 ml", "Beverages", "Soft Drinks", "FizzUp", 40),
    ("P205", "Packaged Water 1 L", "Beverages", "Water", "AquaPure", 20),
    ("P206", "Energy Drink 250 ml", "Beverages", "Energy Drinks", "VoltX", 110),
    ("P207", "Flavoured Milk 200 ml", "Beverages", "Milk Drinks", "CreamyCow", 30),
    ("P301", "Potato Chips 90 g", "Snacks & Confectionery", "Chips", "CrunchTime", 30),
    ("P302", "Instant Noodles 4-pack", "Snacks & Confectionery", "Noodles", "QuickBite", 60),
    ("P303", "Cream Biscuits 300 g", "Snacks & Confectionery", "Biscuits", "BakeJoy", 35),
```

```
    ("P304", "Milk Chocolate 50 g", "Snacks & Confectionery", "Chocolates", "ChocoDelight",
60),
    ("P305", "Chocolate Pouch 100 g", "Snacks & Confectionery", "Chocolates", "ChocoDelight",
120),
    ("P306", "Lollipop Pack", "Snacks & Confectionery", "Candies", "SweetPop", 50),
    ("P307", "Salted Namkeen 400 g", "Snacks & Confectionery", "Namkeen", "SnackHub", 110),
    ("P401", "Shampoo 340 ml", "Personal Care", "Hair Care", "SilkyLocks", 320),
    ("P402", "Conditioner 180 ml", "Personal Care", "Hair Care", "SilkyLocks", 250),
    ("P403", "Body Deodorant 150 ml", "Personal Care", "Deodorants", "ZestMan", 199),
    ("P404", "Face Wash 100 g", "Personal Care", "Skin Care", "GlowFresh", 140),
    ("P405", "Moisturising Lotion 400 ml", "Personal Care", "Skin Care", "SoftSkin", 280),
    ("P406", "Toothpaste 200 g", "Personal Care", "Oral Care", "BrightSmile", 110),
    ("P407", "Bathing Soap 3-pack", "Personal Care", "Soaps", "PureBath", 130),
    ("P501", "Detergent Powder 1 kg", "Home Care", "Laundry", "CleanWave", 130),
    ("P502", "Fabric Conditioner 1 L", "Home Care", "Laundry", "CleanWave", 230),
    ("P503", "Dishwash Liquid 500 ml", "Home Care", "Dishwash", "ShineAll", 110),
    ("P504", "Floor Cleaner 1 L", "Home Care", "Floor Care", "ShineAll", 190),
    ("P505", "Room Freshener 250 ml", "Home Care", "Air Care", "FreshAir", 220),
    ("P506", "Toilet Cleaner 500 ml", "Home Care", "Bathroom Care", "ShineAll", 105),
    ("P507", "Garbage Bags 30 pcs", "Home Care", "Disposables", "TidyHome", 90),
    ("P601", "Full Cream Milk 1 L", "Dairy & Bakery", "Milk", "CreamyCow", 68),
    ("P602", "Curd 400 g", "Dairy & Bakery", "Curd", "CreamyCow", 45),
    ("P603", "Butter 100 g", "Dairy & Bakery", "Butter", "CreamyCow", 62),
    ("P604", "Paneer 200 g", "Dairy & Bakery", "Paneer", "CreamyCow", 95),
    ("P605", "Bread Loaf", "Dairy & Bakery", "Bread", "BakeJoy", 45),
    ("P606", "Cheese Slices 200 g", "Dairy & Bakery", "Cheese", "CreamyCow", 140),
    ("P607", "Eggs (Dozen)", "Dairy & Bakery", "Eggs", "FarmFresh", 84),
]
prod = pd.DataFrame(products, columns=["Product_ID", "Product_Name", "Category",
                                       "Sub_Category", "Brand", "Unit_Price"])
prod["Stock_Available"] = np.random.randint(5, 150, len(prod))

n = 1200
wt = np.random.uniform(0.4, 2.0, len(prod)) / (prod["Unit_Price"] ** 0.3)
idx = np.random.choice(prod.index, size=n, p=wt / wt.sum())
data = prod.loc[idx].reset_index(drop=True)

dates = pd.date_range("2026-07-01", "2026-09-28")
dw = np.where(dates.dayofweek >= 5, 1.5, 1.0)
data["Date"] = np.random.choice(dates, size=n, p=dw / dw.sum())
data["Customer_Type"] = np.random.choice(["Regular", "Occasional", "Bulk Buyer"],
                                         size=n, p=[0.55, 0.33, 0.12])
data["Payment_Method"] = np.random.choice(["UPI", "Cash", "Card"],
                                          size=n, p=[0.48, 0.38, 0.14])
qty = np.random.poisson(1.2, n) + 1
data["Quantity"] = np.where(data["Customer_Type"] == "Bulk Buyer", qty * 3, qty)
data["Discount_Percent"] = np.random.choice([0, 0, 0, 5, 5, 10, 15], size=n).astype(float)
data = data.sort_values("Date").reset_index(drop=True)
data["Bill_ID"] = ["B" + str(1001 + i) for i in range(n)]
data["Record_Source"] = "Simulated"

# Introduce common data-quality problems for the cleaning exercise
raw = data.copy()
raw.loc[raw.sample(30, random_state=1).index, "Discount_Percent"] = np.nan
raw.loc[raw.sample(20, random_state=2).index, "Payment_Method"] = np.nan
i = raw.sample(40, random_state=3).index
raw.loc[i[:15], "Category"] = raw.loc[i[:15], "Category"].str.upper()
raw.loc[i[15:30], "Category"] = raw.loc[i[15:30], "Category"].str.lower()
raw.loc[i[30:], "Category"] = " " + raw.loc[i[30:], "Category"] + " "
raw.loc[raw.sample(5, random_state=4).index, "Quantity"] = -1
raw.loc[raw.sample(4, random_state=5).index, "Unit_Price"] = 0
raw = pd.concat([raw, raw.sample(15, random_state=6)])
raw = raw.sort_values("Bill_ID", kind="stable").reset_index(drop=True)
raw["Date"] = pd.to_datetime(raw["Date"]).dt.strftime("%d-%m-%Y")

cols = ["Bill_ID", "Product_ID", "Product_Name", "Category", "Sub_Category", "Brand",
        "Unit_Price", "Quantity", "Discount_Percent", "Date", "Payment_Method",
        "Customer_Type", "Stock_Available", "Record_Source"]
raw[cols].to_csv("supermarket_sales_raw.csv", index=False)
print("Raw dataset saved:", raw.shape)
```

Output

```
Raw dataset saved: (1215, 14)
```

3. PURPOSE AND SCOPE

3.1 Purpose of the Field Project

- To visit a supermarket and observe how products are displayed and sold.
- To understand what kind of data a supermarket generates through its billing.
- To apply Python programming to analyse supermarket-style sales data.
- To practise data cleaning, processing, analysis and visualisation on a realistic dataset.
- To draw simple, meaningful conclusions from the analysis.

3.2 Scope

- Analysis of sales, quantity, category, product, brand, price, discount, payment method, customer type and date.
- Use of pandas, NumPy, Matplotlib and Seaborn in Python.
- The project does not include profit analysis, because cost price data is not available.
- The project does not include prediction or machine-learning models.

4. FIELD VISIT DESCRIPTION

The observations given below are based on the geotagged photographs taken during the visit on 29/09/2026 (Figures 1 to 3).

4.1 Purpose of the Visit

The purpose of the visit was to see a supermarket at work and to understand how the sales data used in this project is generated in real life, from shelf display to the billing counter.

4.2 What Was Observed

- The shop signboard reads “Choice Super Market” and shows an address in Virar (E) (Figure 1).
- The entrance has products displayed both inside and on the outer frame, including hanging items on the right side, and a glass-door chiller at the entrance (Figure 1).
- Shelves are fully stocked from floor to ceiling with a large variety of products (Figure 2).
- A computer with a keyboard and a scanner-like device is kept on the billing counter (Figure 3).

4.3 Product Categories Available

From the photographs, the following types of products can be seen: personal care items (deodorants, shampoos, conditioners, face washes and lotions), home care items (detergent, fabric conditioner and air fresheners), beverages (packaged juices, soft drinks and energy drinks), chocolates and confectionery, and small packaged items hanging near the counter. The categories used in the dataset were chosen to represent such product types.

4.4 Arrangement of Products

Products of the same type are placed together on shelves. In Figure 2, deodorants, hair care, skin care and air fresheners are kept on separate shelves, while large bottles of home care products are kept on the topmost shelf. Chocolate pouches are hung on a rack at the side.

Handwritten price marks on the bottles and a handwritten offer notice near the deodorants are also visible.

4.5 Customer Purchasing Environment

The shop is a compact, closely stocked space in which customers can see many products at close distance. Products are displayed at different heights, and offers and price marks are placed near the items, which can influence what customers choose.

4.6 Billing / Counter Observations

Figure 3 shows the billing counter, where a person is seated at a computer with a keyboard. Beverages, packaged drinks and small items are kept close to the counter, and hanging packets are displayed on the wall beside it. The billing computer is the point where each sale can be recorded as data.

4.7 Importance of Data in Supermarket Management

Since each bill records product, quantity, price and date, the collected data can be used to decide stock quantities, arrange shelves and plan discounts. This project demonstrates how such data can be analysed with Python.

5. GEOTAGGED PHOTOGRAPHS OF THE FIELD VISIT

The photographs below are inserted as taken. The location, date and time shown on the photographs are as recorded by the GPS Map Camera application and have not been altered.



Figure 1: Entrance of Choice Super Market, Virar (East)

The photograph shows the signboard and entrance of the supermarket. The geotag on the photograph shows Vasai-Virar, Maharashtra, Lat 19.467296°, Long 72.815367°, Tuesday 29/09/2026, 09:36 PM (GMT +05:30). It demonstrates the location of the field visit and the outer display of products.

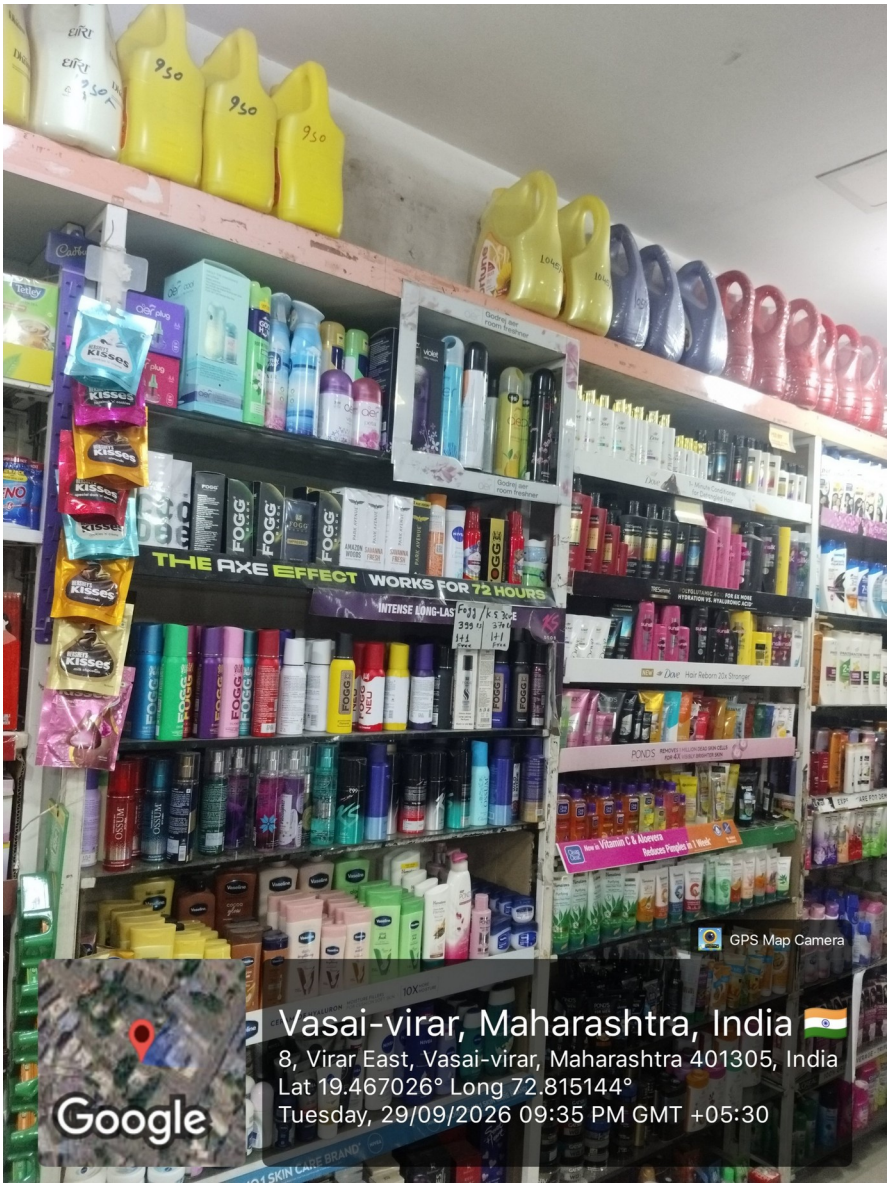


Figure 2: Product shelves inside the supermarket

The photograph shows shelves of personal care and home care products, with chocolate pouches on a hanging rack. The geotag shows Lat 19.467026°, Long 72.815144°, Tuesday 29/09/2026, 09:35 PM (GMT +05:30). It demonstrates the grouping of products by type and the variety of items sold.



Figure 3: Billing counter of the supermarket

The photograph shows the billing counter with a computer and keyboard, beverages and small items around it. The geotag shows Lat 19.466971°, Long 72.81509°, Tuesday 29/09/2026, 09:32 PM (GMT +05:30). It demonstrates where sales transactions are recorded.

6. OBJECTIVES OF THE FIELD PROJECT

1. To understand the role of data analytics in supermarkets.
2. To analyse supermarket product and sales data.
3. To clean and preprocess the dataset using Python.
4. To identify sales patterns.
5. To compare product categories.
6. To identify high-performing products.
7. To study pricing and discount patterns.
8. To summarise the dataset statistically.
9. To visualise important trends using graphs.
10. To derive useful conclusions from the analysis.

7. TECHNOLOGIES AND TOOLS USED

Python: A simple and widely used programming language. All analysis in this project was done in Python.

Jupyter Notebook / Google Colab: Environments in which Python code can be written and run in cells and the output is shown just below the code.

Pandas: A library used to load, clean, group and summarise tabular data using DataFrames.

NumPy: A library for numerical calculations and arrays; used here for conditions and numeric operations.

Matplotlib: A library used to draw graphs such as bar charts, histograms, line graphs and scatter plots.

Seaborn: A library built on Matplotlib that makes attractive statistical graphs such as boxplots and heatmaps.

8. DATASET DESCRIPTION

Note: Run the loading code of Section 9 (Step 1) first, so that the DataFrame df is available.

Number of rows and columns

Command / Code

```
print("Number of rows    :", df.shape[0])
print("Number of columns :", df.shape[1])
```

Output

```
Number of rows    : 1215
Number of columns : 14
```

Column description table

Command / Code

```
col_info = pd.DataFrame({
    "Column": ["Bill_ID", "Product_ID", "Product_Name", "Category", "Sub_Category",
               "Brand", "Unit_Price", "Quantity", "Discount_Percent", "Date",
               "Payment_Method", "Customer_Type", "Stock_Available", "Record_Source"],
    "Type": ["Text", "Text", "Text", "Text", "Text", "Text", "Decimal", "Integer",
             "Decimal", "Text->Date", "Text", "Text", "Integer", "Text"],
    "Description": ["Bill / transaction number", "Unique product code",
                    "Product name with pack size", "Main product category",
                    "Product sub-category", "Brand name (fictional)",
                    "Price of one unit (Rs.)", "Units sold in the bill line",
                    "Discount given (%)", "Date of sale (dd-mm-yyyy)",
                    "Cash / UPI / Card", "Regular / Occasional / Bulk Buyer",
                    "Units in stock (simulated)", "Marks record as simulated"]})
print(f"{'Column':<18}{'Type':<12}Description")
print("-" * 70)
for _, r in col_info.iterrows():
    print(f"{r['Column']:<18}{r['Type']:<12}{r['Description']}")
```

Output

```
Column            Type        Description
----------------------------------------------------------------------
Bill_ID           Text        Bill / transaction number
Product_ID        Text        Unique product code
Product_Name      Text        Product name with pack size
Category          Text        Main product category
Sub_Category      Text        Product sub-category
Brand             Text        Brand name (fictional)
Unit_Price        Decimal     Price of one unit (Rs.)
Quantity          Integer     Units sold in the bill line
Discount_Percent  Decimal     Discount given (%)
Date              Text->Date  Date of sale (dd-mm-yyyy)
Payment_Method    Text        Cash / UPI / Card
Customer_Type     Text        Regular / Occasional / Bulk Buyer
Stock_Available   Integer     Units in stock (simulated)
Record_Source     Text        Marks record as simulated
```

Dataset information and data types

Command / Code

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 1215 entries, 0 to 1214
Data columns (total 14 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Bill_ID           1215 non-null   str
```

```
 1   Product_ID        1215 non-null   str
 2   Product_Name      1215 non-null   str
 3   Category          1215 non-null   str
 4   Sub_Category      1215 non-null   str
 5   Brand             1215 non-null   str
 6   Unit_Price        1215 non-null   int64
 7   Quantity          1215 non-null   int64
 8   Discount_Percent  1185 non-null   float64
 9   Date              1215 non-null   str
 10  Payment_Method    1195 non-null   str
 11  Customer_Type     1215 non-null   str
 12  Stock_Available   1215 non-null   int64
 13  Record_Source     1215 non-null   str
dtypes: float64(1), int64(3), str(10)
memory usage: 133.0 KB
```

Sample records (random 5 rows)

Command / Code

```
print(df.sample(5, random_state=1))
```

Output

```
     Bill_ID Product_ID           Product_Name                Category   Sub_Category  \
838    B1826       P303   Cream Biscuits 300 g  Snacks & Confectionery       Biscuits
776    B1765       P206    Energy Drink 250 ml               Beverages  Energy Drinks
341    B1333       P501  Detergent Powder 1 kg               Home Care        Laundry
1003   B1990       P107       Tea Powder 500 g       Staples & Grocery            Tea
186    B1184       P403  Body Deodorant 150 ml           Personal Care     Deodorants

            Brand  Unit_Price  Quantity  Discount_Percent        Date Payment_Method  \
838       BakeJoy          35         3               0.0  30-08-2026            UPI
776         VoltX         110         1               5.0  27-08-2026           Cash
341     CleanWave         130         3               5.0  26-07-2026           Cash
1003  MorningBrew         240         1               0.0  12-09-2026            UPI
186       ZestMan         199         3               0.0  13-07-2026            UPI

     Customer_Type  Stock_Available Record_Source
838     Occasional               92     Simulated
776     Occasional              108     Simulated
341     Bulk Buyer               59     Simulated
1003       Regular              107     Simulated
186     Occasional               53     Simulated
```

Basic statistical description (raw data)

Command / Code

```
print(df.describe().round(2))
```

Output

```
       Unit_Price  Quantity  Discount_Percent  Stock_Available
count     1215.00   1215.00           1185.00          1215.00
mean       103.76      2.64              4.89            77.81
std         79.45      1.90              5.37            41.08
min          0.00     -1.00              0.00             6.00
25%         45.00      2.00              0.00            42.00
50%         84.00      2.00              5.00            79.00
75%        130.00      3.00             10.00           107.00
max        620.00     15.00             15.00           139.00
```

9. DATA LOADING

Importing libraries and loading the dataset

Command / Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

pd.set_option("display.width", 92)
pd.set_option("display.max_columns", 30)

df = pd.read_csv("supermarket_sales_raw.csv")
print("Dataset loaded successfully")
```

Output

```
Dataset loaded successfully
```

Displaying the first 5 rows

Command / Code

```
print(df.head())
```

Output

```
  Bill_ID Product_ID            Product_Name                Category Sub_Category  \
0   B1001       P302  Instant Noodles 4-pack  Snacks & Confectionery      Noodles
1   B1002       P402      Conditioner 180 ml          Personal Care    Hair Care
2   B1003       P104       Sunflower Oil 1 L       Staples & Grocery   Edible Oil
3   B1004       P502  Fabric Conditioner 1 L              Home Care      Laundry
4   B1005       P203             Cola 750 ml              Beverages  Soft Drinks

        Brand  Unit_Price  Quantity  Discount_Percent        Date Payment_Method  \
0   QuickBite          60         2               0.0  01-07-2026           Cash
1  SilkyLocks         250         2              15.0  01-07-2026           Card
2     SunDrop         145         4               0.0  01-07-2026           Card
3   CleanWave         230         2              15.0  01-07-2026            UPI
4      FizzUp          40         3               0.0  01-07-2026           Card

  Customer_Type  Stock_Available Record_Source
0    Occasional                6     Simulated
1       Regular               93     Simulated
2    Occasional              111     Simulated
3       Regular               68     Simulated
4       Regular               92     Simulated
```

Displaying the last 5 rows

Command / Code

```
print(df.tail())
```

Output

```
     Bill_ID Product_ID            Product_Name                Category   Sub_Category  \
1210   B2196       P605              Bread Loaf          Dairy & Bakery          Bread
1211   B2197       P302  Instant Noodles 4-pack  Snacks & Confectionery        Noodles
1212   B2198       P206     Energy Drink 250 ml              Beverages  Energy Drinks
1213   B2199       P201        Orange Juice 1 L              Beverages         Juices
1214   B2200       P503  Dishwash Liquid 500 ml              Home Care       Dishwash

           Brand  Unit_Price  Quantity  Discount_Percent        Date Payment_Method  \
1210     BakeJoy          45         3               5.0  28-09-2026           Card
1211   QuickBite          60         6              15.0  28-09-2026            UPI
1212       VoltX         110         2               0.0  28-09-2026            NaN
```

```
1213  FruitFresh         110         2              10.0  28-09-2026           Cash
1214    ShineAll         110         1               5.0  28-09-2026           Cash

     Customer_Type  Stock_Available Record_Source
1210       Regular               18     Simulated
1211    Bulk Buyer                6     Simulated
1212       Regular              108     Simulated
1213       Regular              126     Simulated
1214    Occasional              135     Simulated
```

Checking the shape of the dataset

Command / Code

```
print(df.shape)
```

Output

```
(1215, 14)
```

Checking the column names

Command / Code

```
print(df.columns.tolist())
```

Output

```
['Bill_ID', 'Product_ID', 'Product_Name', 'Category', 'Sub_Category', 'Brand', 'Unit_Price',
'Quantity', 'Discount_Percent', 'Date', 'Payment_Method', 'Customer_Type', 'Stock_Available',
'Record_Source']
```

Checking the data types

Command / Code

```
print(df.dtypes)
```

Output

```
Bill_ID                str
Product_ID             str
Product_Name           str
Category               str
Sub_Category           str
Brand                  str
Unit_Price           int64
Quantity             int64
Discount_Percent   float64
Date                   str
Payment_Method         str
Customer_Type          str
Stock_Available      int64
Record_Source          str
dtype: object
```

Checking missing values

Command / Code

```
print(df.isnull().sum())
```

Output

```
Bill_ID             0
Product_ID          0
Product_Name        0
Category            0
Sub_Category        0
```

```
Brand                0
Unit_Price           0
Quantity             0
Discount_Percent    30
Date                 0
Payment_Method      20
Customer_Type        0
Stock_Available      0
Record_Source        0
dtype: int64
```

10. DATA CLEANING

Step 1: Checking missing values

Command / Code

```
missing = df.isnull().sum()
print(pd.DataFrame({"Missing": missing,
                    "Percent": (missing / len(df) * 100).round(2)}))
```

Output

```
                  Missing  Percent
Bill_ID                 0     0.00
Product_ID              0     0.00
Product_Name            0     0.00
Category                0     0.00
Sub_Category            0     0.00
Brand                   0     0.00
Unit_Price              0     0.00
Quantity                0     0.00
Discount_Percent       30     2.47
Date                    0     0.00
Payment_Method         20     1.65
Customer_Type           0     0.00
Stock_Available         0     0.00
Record_Source           0     0.00
```

Step 2: Handling missing values

Command / Code

```
df["Discount_Percent"] = df["Discount_Percent"].fillna(0)
df["Payment_Method"] = df["Payment_Method"].fillna("Not Recorded")
print(df.isnull().sum().sum(), "missing values remaining")
```

Output

```
0 missing values remaining
```

Step 3: Removing duplicate records

Command / Code

```
print("Duplicate rows before:", df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicate rows after :", df.duplicated().sum())
print("Shape after removing duplicates:", df.shape)
```

Output

```
Duplicate rows before: 15
Duplicate rows after : 0
Shape after removing duplicates: (1200, 14)
```

Step 4: Handling inconsistent category names

Command / Code

```
print("Before cleaning:")
print(df["Category"].value_counts())
df["Category"] = df["Category"].str.strip().str.title()
print("\nAfter cleaning:")
print(df["Category"].value_counts())
```

Output

```
Before cleaning:
Category
```

```
Snacks & Confectionery      255
Beverages                   228
Dairy & Bakery              225
Home Care                   162
Staples & Grocery           156
Personal Care               134
beverages                     5
staples & grocery             5
home care                     4
DAIRY & BAKERY                4
PERSONAL CARE                 3
BEVERAGES                     3
 Staples & Grocery            2
 Beverages                    2
 Dairy & Bakery               2
HOME CARE                     2
 Snacks & Confectionery       2
SNACKS & CONFECTIONERY        2
 Personal Care                1
STAPLES & GROCERY             1
snacks & confectionery        1
 Home Care                    1
Name: count, dtype: int64

After cleaning:
Category
Snacks & Confectionery    260
Beverages                 238
Dairy & Bakery            231
Home Care                 169
Staples & Grocery         164
Personal Care             138
Name: count, dtype: int64
```

Step 5: Correcting data types

Command / Code

```
df["Date"] = pd.to_datetime(df["Date"], format="%d-%m-%Y")
df["Quantity"] = df["Quantity"].astype(int)
print(df.dtypes)
```

Output

```
Bill_ID                       str
Product_ID                    str
Product_Name                  str
Category                      str
Sub_Category                  str
Brand                         str
Unit_Price                  int64
Quantity                    int64
Discount_Percent          float64
Date                datetime64[us]
Payment_Method                str
Customer_Type                 str
Stock_Available             int64
Record_Source                 str
dtype: object
```

Step 6: Checking and removing invalid values

Command / Code

```
print("Quantity <= 0     :", (df["Quantity"] <= 0).sum())
print("Unit_Price <= 0   :", (df["Unit_Price"] <= 0).sum())
print("Discount outside 0-100 :", (~df["Discount_Percent"].between(0, 100)).sum())
print()
print(df[(df["Quantity"] <= 0) | (df["Unit_Price"] <= 0)][
    ["Bill_ID", "Product_Name", "Unit_Price", "Quantity"]])
df = df[(df["Quantity"] > 0) & (df["Unit_Price"] > 0)]
print("\nShape after removing invalid rows:", df.shape)
```

Output

```
Quantity <= 0     : 5
Unit_Price <= 0   : 4
Discount outside 0-100 : 0

     Bill_ID            Product_Name  Unit_Price  Quantity
11     B1012              Bread Loaf           0         1
165    B1163   Body Deodorant 150 ml         199        -1
242    B1236      Packaged Water 1 L           0         2
421    B1412        Toothpaste 200 g         110        -1
716    B1705  Dishwash Liquid 500 ml         110        -1
901    B1888   Body Deodorant 150 ml           0         1
922    B1909    Cream Biscuits 300 g          35        -1
998    B1985              Cola 750 ml           0         1
1066   B2052   Detergent Powder 1 kg         130        -1

Shape after removing invalid rows: (1191, 14)
```

Step 7: Dropping unnecessary column and saving the cleaned dataset

Command / Code

```
df = df.drop(columns=["Record_Source"]).reset_index(drop=True)
df.to_csv("supermarket_sales_cleaned.csv", index=False)
print("Cleaned dataset shape:", df.shape)
print(df.columns.tolist())
```

Output

```
Cleaned dataset shape: (1191, 13)
['Bill_ID', 'Product_ID', 'Product_Name', 'Category', 'Sub_Category', 'Brand', 'Unit_Price',
'Quantity', 'Discount_Percent', 'Date', 'Payment_Method', 'Customer_Type', 'Stock_Available']
```

11. DATA PROCESSING / FEATURE ENGINEERING

Profit is not calculated because cost price data is not available in the dataset.

Creating Total Sales, Discount Amount and Final Sales

Command / Code

```
df["Total_Sales"] = df["Unit_Price"] * df["Quantity"]
df["Discount_Amount"] = (df["Total_Sales"] * df["Discount_Percent"] / 100).round(2)
df["Final_Sales"] = (df["Total_Sales"] - df["Discount_Amount"]).round(2)
print(df[["Product_Name", "Unit_Price", "Quantity", "Discount_Percent",
          "Total_Sales", "Discount_Amount", "Final_Sales"]].head())
```

Output

```
             Product_Name  Unit_Price  Quantity  Discount_Percent  Total_Sales  \
0  Instant Noodles 4-pack          60         2               0.0          120
1      Conditioner 180 ml         250         2              15.0          500
2       Sunflower Oil 1 L         145         4               0.0          580
3  Fabric Conditioner 1 L         230         2              15.0          460
4             Cola 750 ml          40         3               0.0          120

   Discount_Amount  Final_Sales
0              0.0        120.0
1             75.0        425.0
2              0.0        580.0
3             69.0        391.0
4              0.0        120.0
```

Creating date-based variables

Command / Code

```
df["Month"] = df["Date"].dt.month_name()
df["Month_No"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["Day_Name"] = df["Date"].dt.day_name()
df["Day_Type"] = np.where(df["Day_Name"].isin(["Saturday", "Sunday"]),
                          "Weekend", "Weekday")
print(df[["Date", "Month", "Month_No", "Day", "Day_Name", "Day_Type"]].head())
```

Output

```
        Date Month  Month_No  Day   Day_Name Day_Type
0 2026-07-01  July         7    1  Wednesday  Weekday
1 2026-07-01  July         7    1  Wednesday  Weekday
2 2026-07-01  July         7    1  Wednesday  Weekday
3 2026-07-01  July         7    1  Wednesday  Weekday
4 2026-07-01  July         7    1  Wednesday  Weekday
```

Creating Sales Category and Quantity Category

Command / Code

```
df["Sales_Category"] = pd.cut(df["Final_Sales"], bins=[0, 100, 300, 700, np.inf],
                              labels=["Low", "Medium", "High", "Very High"])
df["Quantity_Category"] = pd.cut(df["Quantity"], bins=[0, 1, 3, np.inf],
                                 labels=["Single", "Small (2-3)", "Bulk (4+)"])
print(df["Sales_Category"].value_counts())
print()
print(df["Quantity_Category"].value_counts())
```

Output

```
Sales_Category
Medium       487
Low          370
High         263
Very High     71
Name: count, dtype: int64

Quantity_Category
Small (2-3)    700
Single         292
Bulk (4+)      199
Name: count, dtype: int64
```

Checking the processed dataset

Command / Code

```
print(df.shape)
print(df.columns.tolist())
print(df.head(3).T)
```

Output

```
(1191, 23)
['Bill_ID', 'Product_ID', 'Product_Name', 'Category', 'Sub_Category', 'Brand', 'Unit_Price',
'Quantity', 'Discount_Percent', 'Date', 'Payment_Method', 'Customer_Type', 'Stock_Available',
'Total_Sales', 'Discount_Amount', 'Final_Sales', 'Month', 'Month_No', 'Day', 'Day_Name',
'Day_Type', 'Sales_Category', 'Quantity_Category']
                                      0                    1                    2
Bill_ID                           B1001                B1002                B1003
Product_ID                         P302                 P402                 P104
Product_Name       Instant Noodles 4-pack   Conditioner 180 ml    Sunflower Oil 1 L
Category           Snacks & Confectionery        Personal Care    Staples & Grocery
Sub_Category                      Noodles            Hair Care          Edible Oil
Brand                           QuickBite           SilkyLocks             SunDrop
Unit_Price                             60                  250                  145
Quantity                                2                    2                    4
Discount_Percent                      0.0                 15.0                  0.0
Date                  2026-07-01 00:00:00  2026-07-01 00:00:00  2026-07-01 00:00:00
Payment_Method                       Cash                 Card                 Card
Customer_Type                  Occasional              Regular           Occasional
Stock_Available                         6                   93                  111
Total_Sales                           120                  500                  580
Discount_Amount                       0.0                  75.0                 0.0
Final_Sales                         120.0                425.0                580.0
Month                                July                 July                 July
Month_No                                7                    7                    7
Day                                     1                    1                    1
Day_Name                        Wednesday            Wednesday            Wednesday
Day_Type                          Weekday              Weekday              Weekday
Sales_Category                     Medium                 High                 High
Quantity_Category             Small (2-3)          Small (2-3)           Bulk (4+)
```

12. EXPLORATORY DATA ANALYSIS

1. Total products, records and quantity sold

Command / Code

```
print("Total records (bill lines) :", len(df))
print("Total unique products      :", df["Product_ID"].nunique())
print("Total categories           :", df["Category"].nunique())
print("Total brands               :", df["Brand"].nunique())
print("Total quantity sold        :", df["Quantity"].sum())
```

Output

```
Total records (bill lines) : 1191
Total unique products      : 42
Total categories           : 6
Total brands               : 28
Total quantity sold        : 3160
```

2. Total, average, minimum and maximum sales

Command / Code

```
print("Total Sales (before discount)  : Rs.", round(df["Total_Sales"].sum(), 2))
print("Total Discount Given           : Rs.", round(df["Discount_Amount"].sum(), 2))
print("Total Final Sales              : Rs.", round(df["Final_Sales"].sum(), 2))
print("Average Final Sales per record : Rs.", round(df["Final_Sales"].mean(), 2))
print("Minimum Final Sales            : Rs.", df["Final_Sales"].min())
print("Maximum Final Sales            : Rs.", df["Final_Sales"].max())
```

Output

```
Total Sales (before discount)  : Rs. 317577
Total Discount Given           : Rs. 14812.1
Total Final Sales              : Rs. 302764.9
Average Final Sales per record : Rs. 254.21
Minimum Final Sales            : Rs. 18.0
Maximum Final Sales            : Rs. 2388.0
```

3. Most frequently occurring category

Command / Code

```
print(df["Category"].value_counts())
print("\nMost frequent category:", df["Category"].mode()[0])
```

Output

```
Category
Snacks & Confectionery    259
Beverages                 236
Dairy & Bakery            230
Home Care                 167
Staples & Grocery         164
Personal Care             135
Name: count, dtype: int64

Most frequent category: Snacks & Confectionery
```

4. Best-selling products (by Final Sales)

Command / Code

```
prod_sales = df.groupby("Product_Name")["Final_Sales"].sum().sort_values(ascending=False)
print(prod_sales.head(10).round(2))
```

Output

```
Product_Name
Moisturising Lotion 400 ml    27048.00
Tea Powder 500 g              21324.00
Wheat Flour 5 kg              21203.00
Room Freshener 250 ml         19789.00
Body Deodorant 150 ml         13313.10
Chocolate Pouch 100 g         13236.00
Conditioner 180 ml            11150.00
Fabric Conditioner 1 L        10511.00
Paneer 200 g                  10036.75
Toor Dal 1 kg                  9834.00
Name: Final_Sales, dtype: float64
```

5. Best-selling products (by quantity)

Command / Code

```
prod_qty = df.groupby("Product_Name")["Quantity"].sum().sort_values(ascending=False)
print(prod_qty.head(10))
```

Output

```
Product_Name
Flavoured Milk 200 ml     174
Packaged Water 1 L        168
Potato Chips 90 g         164
Bread Loaf                142
Butter 100 g              138
Cream Biscuits 300 g      123
Instant Noodles 4-pack    123
Curd 400 g                121
Chocolate Pouch 100 g     115
Paneer 200 g              112
Name: Quantity, dtype: int64
```

6. Lowest-selling products (by Final Sales)

Command / Code

```
print(prod_sales.tail(10).sort_values().round(2))
```

Output

```
Product_Name
Iodised Salt 1 kg      1197.30
Full Cream Milk 1 L    1482.40
Cheese Slices 200 g    1498.00
Lollipop Pack          1625.00
Basmati Rice 5 kg      1767.00
Lemon Soda 750 ml      2258.00
Shampoo 340 ml         2480.00
Sunflower Oil 1 L      2718.75
Floor Cleaner 1 L      3154.00
Packaged Water 1 L     3198.00
Name: Final_Sales, dtype: float64
```

7. Category-wise sales

Command / Code

```
cat_sales = df.groupby("Category")["Final_Sales"].sum().sort_values(ascending=False)
print(cat_sales.round(2))
```

Output

```
Category
Personal Care             70236.10
Staples & Grocery         62671.25
Home Care                 60029.00
Snacks & Confectionery    38653.50
Dairy & Bakery            38058.30
Beverages                 33116.75
Name: Final_Sales, dtype: float64
```

8. Brand-wise sales (top 10)

Command / Code

```
brand_sales = df.groupby("Brand")["Final_Sales"].sum().sort_values(ascending=False)
print(brand_sales.head(10).round(2))
```

Output

```
Brand
CreamyCow       31435.65
SoftSkin        27048.00
MorningBrew     21324.00
HarvestPure     21203.00
ShineAll        19840.50
FreshAir        19789.00
ChocoDelight    17709.00
CleanWave       15607.00
FruitFresh      14980.75
SilkyLocks      13630.00
Name: Final_Sales, dtype: float64
```

9. Price distribution

Command / Code

```
print(df["Unit_Price"].describe().round(2))
price_band = pd.cut(df["Unit_Price"], bins=[0, 50, 100, 200, 400, 1000],
                    labels=["Rs. 0-50", "Rs. 51-100", "Rs. 101-200",
                            "Rs. 201-400", "Rs. 401-1000"])
print()
print(price_band.value_counts().sort_index())
```

Output

```
count    1191.00
mean      103.92
std        79.48
min        20.00
25%        45.00
50%        84.00
75%       130.00
max       620.00
Name: Unit_Price, dtype: float64

Unit_Price
Rs. 0-50        437
Rs. 51-100      229
Rs. 101-200     332
Rs. 201-400     191
Rs. 401-1000      2
Name: count, dtype: int64
```

10. Quantity distribution

Command / Code

```
print(df["Quantity"].value_counts().sort_index())
```

Output

```
Quantity
1     292
2     409
3     291
4      80
5      22
6      57
9      26
12     13
15      1
Name: count, dtype: int64
```

11. Discount analysis

Command / Code

```
print(df["Discount_Percent"].value_counts().sort_index())
print()
print(df.groupby("Category")["Discount_Amount"].sum().round(2))
```

Output

```
Discount_Percent
0.0     547
5.0     326
10.0    148
15.0    170
Name: count, dtype: int64

Category
Beverages                1648.25
Dairy & Bakery           2208.70
Home Care                2766.00
Personal Care            3083.90
Snacks & Confectionery   1821.50
Staples & Grocery        3283.75
Name: Discount_Amount, dtype: float64
```

12. Payment-method analysis

Command / Code

```
pay = df.groupby("Payment_Method").agg(Records=("Bill_ID", "count"),
                                       Final_Sales=("Final_Sales", "sum")).round(2)
print(pay.sort_values("Records", ascending=False))
```

Output

```
                Records  Final_Sales
Payment_Method
UPI                 581    156384.60
Cash                436    108751.15
Card                154     33933.95
Not Recorded         20      3695.20
```

13. Customer-type analysis

Command / Code

```
cust = df.groupby("Customer_Type").agg(Records=("Bill_ID", "count"),
                                       Avg_Quantity=("Quantity", "mean"),
                                       Avg_Final_Sales=("Final_Sales", "mean")).round(2)
print(cust)
```

Output

```
               Records  Avg_Quantity  Avg_Final_Sales
Customer_Type
Bulk Buyer         134          6.27           571.24
Occasional         405          2.22           222.02
Regular            652          2.18           209.05
```

14. Month-wise sales

Command / Code

```
print(df.groupby(["Month_No", "Month"])["Final_Sales"].sum().round(2))
```

Output

```
Month_No  Month
7         July          97200.30
8         August       119633.75
9         September     85930.85
Name: Final_Sales, dtype: float64
```

13. DATA SUMMARIZATION

Summary statistics using describe()

Command / Code

```
num_cols = ["Unit_Price", "Quantity", "Discount_Percent", "Total_Sales",
            "Discount_Amount", "Final_Sales"]
print(df[num_cols].describe().round(2))
```

Output

```
       Unit_Price  Quantity  Discount_Percent  Total_Sales  Discount_Amount  Final_Sales
count     1191.00   1191.00           1191.00      1191.00          1191.00      1191.00
mean       103.92      2.65              4.75       266.65            12.44       254.21
std         79.48      1.89              5.37       282.56            24.19       271.07
min         20.00      1.00              0.00        20.00             0.00        18.00
25%         45.00      2.00              0.00        90.00             0.00        89.25
50%         84.00      2.00              5.00       180.00             3.00       168.00
75%        130.00      3.00             10.00       330.00            15.75       325.88
max        620.00     15.00             15.00      2388.00           234.00      2388.00
```

Category-wise summary using groupby() and agg()

Command / Code

```
cat_summary = df.groupby("Category").agg(
    Records=("Bill_ID", "count"),
    Qty_Sold=("Quantity", "sum"),
    Total_Sales=("Total_Sales", "sum"),
    Final_Sales=("Final_Sales", "sum"),
    Avg_Sale=("Final_Sales", "mean")).round(2)
print(cat_summary.sort_values("Final_Sales", ascending=False))
```

Output

```
                      Records  Qty_Sold  Total_Sales  Final_Sales  Avg_Sale
Category
Personal Care             135       361        73320     70236.10    520.27
Staples & Grocery         164       415        65955     62671.25    382.14
Home Care                 167       420        62795     60029.00    359.46
Snacks & Confectionery    259       670        40475     38653.50    149.24
Dairy & Bakery            230       620        40267     38058.30    165.47
Beverages                 236       674        34765     33116.75    140.33
```

Product-wise summary (top 10 by Final Sales)

Command / Code

```
prod_summary = df.groupby("Product_Name").agg(
    Qty_Sold=("Quantity", "sum"),
    Final_Sales=("Final_Sales", "sum")).round(2)
print(prod_summary.sort_values("Final_Sales", ascending=False).head(10))
```

Output

```
                          Qty_Sold  Final_Sales
Product_Name
Moisturising Lotion 400 ml       100     27048.00
Tea Powder 500 g                  93     21324.00
Wheat Flour 5 kg                  87     21203.00
Room Freshener 250 ml             93     19789.00
Body Deodorant 150 ml             70     13313.10
Chocolate Pouch 100 g            115     13236.00
Conditioner 180 ml                48     11150.00
Fabric Conditioner 1 L            48     10511.00
Paneer 200 g                     112     10036.75
Toor Dal 1 kg                     62      9834.00
```

Brand-wise summary (top 10 by Final Sales)

Command / Code

```
brand_summary = df.groupby("Brand").agg(
    Products=("Product_ID", "nunique"),
    Qty_Sold=("Quantity", "sum"),
    Final_Sales=("Final_Sales", "sum")).round(2)
print(brand_summary.sort_values("Final_Sales", ascending=False).head(10))
```

Output

```
             Products  Qty_Sold  Final_Sales
Brand
CreamyCow            6       579     31435.65
SoftSkin             1       100     27048.00
MorningBrew          1        93     21324.00
HarvestPure          1        87     21203.00
ShineAll             3       182     19840.50
FreshAir             1        93     19789.00
ChocoDelight         2       193     17709.00
CleanWave            2        89     15607.00
FruitFresh           2       139     14980.75
SilkyLocks           2        56     13630.00
```

Monthly summary

Command / Code

```
monthly_summary = df.groupby(["Month_No", "Month"]).agg(
    Records=("Bill_ID", "count"),
    Qty_Sold=("Quantity", "sum"),
    Final_Sales=("Final_Sales", "sum"),
    Avg_Sale=("Final_Sales", "mean")).round(2)
print(monthly_summary)
```

Output

```
                  Records  Qty_Sold  Final_Sales  Avg_Sale
Month_No Month
7        July         396      1019     97200.30    245.46
8        August       438      1207    119633.75    273.14
9        September    357       934     85930.85    240.70
```

Pivot table: Category-wise sales for each month

Command / Code

```
pivot_cat_month = pd.pivot_table(df, values="Final_Sales", index="Category",
                                 columns="Month", aggfunc="sum", fill_value=0)
print(pivot_cat_month[["July", "August", "September"]].round(0))
```

Output

```
Month                     July   August  September
Category
Beverages              10062.0  12086.0    10968.0
Dairy & Bakery         12295.0  13981.0    11782.0
Home Care              21922.0  20970.0    17137.0
Personal Care          24953.0  28266.0    17018.0
Snacks & Confectionery 11798.0  16414.0    10442.0
Staples & Grocery      16170.0  27917.0    18584.0
```

Payment-method and customer-type distribution using value_counts()

Command / Code

```
print((df["Payment_Method"].value_counts(normalize=True) * 100).round(2))
print()
print((df["Customer_Type"].value_counts(normalize=True) * 100).round(2))
print()
print(pd.crosstab(df["Customer_Type"], df["Payment_Method"]))
```

Output

```
Payment_Method
UPI             48.78
Cash            36.61
Card            12.93
Not Recorded     1.68
Name: proportion, dtype: float64

Customer_Type
Regular       54.74
Occasional    34.01
Bulk Buyer    11.25
Name: proportion, dtype: float64

Payment_Method  Card  Cash  Not Recorded  UPI
Customer_Type
Bulk Buyer        15    50             3   66
Occasional        49   156             7  193
Regular           90   230            10  322
```

Weekday versus weekend summary

Command / Code

```
day_summary = df.groupby("Day_Type").agg(Records=("Bill_ID", "count"),
                                         Final_Sales=("Final_Sales", "sum"),
                                         Avg_Sale=("Final_Sales", "mean")).round(2)
print(day_summary)
```

Output

```
          Records  Final_Sales  Avg_Sale
Day_Type
Weekday       737    181332.25    246.04
Weekend       454    121432.65    267.47
```

14. DATA VISUALIZATION

Figure 4: Total Final Sales by Product Category

Command / Code

```
sns.set_theme(style="whitegrid")
cat_sales = df.groupby("Category")["Final_Sales"].sum().sort_values(ascending=False)
plt.figure(figsize=(9, 5.5))
sns.barplot(x=cat_sales.index, y=cat_sales.values, hue=cat_sales.index,
            palette="viridis", legend=False)
plt.title("Figure 4: Total Final Sales by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Total Final Sales (Rs.)")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```

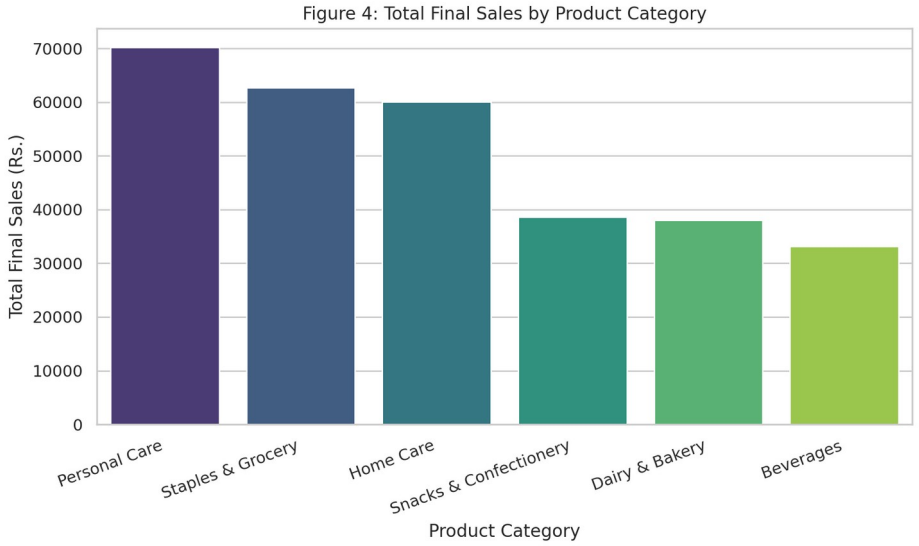


Figure 5: Total Quantity Sold by Category

Command / Code

```
cat_qty = df.groupby("Category")["Quantity"].sum().sort_values(ascending=False)
plt.figure(figsize=(9, 5.5))
sns.barplot(x=cat_qty.index, y=cat_qty.values, hue=cat_qty.index,
            palette="mako", legend=False)
plt.title("Figure 5: Total Quantity Sold by Category")
plt.xlabel("Product Category")
plt.ylabel("Total Quantity Sold (units)")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```

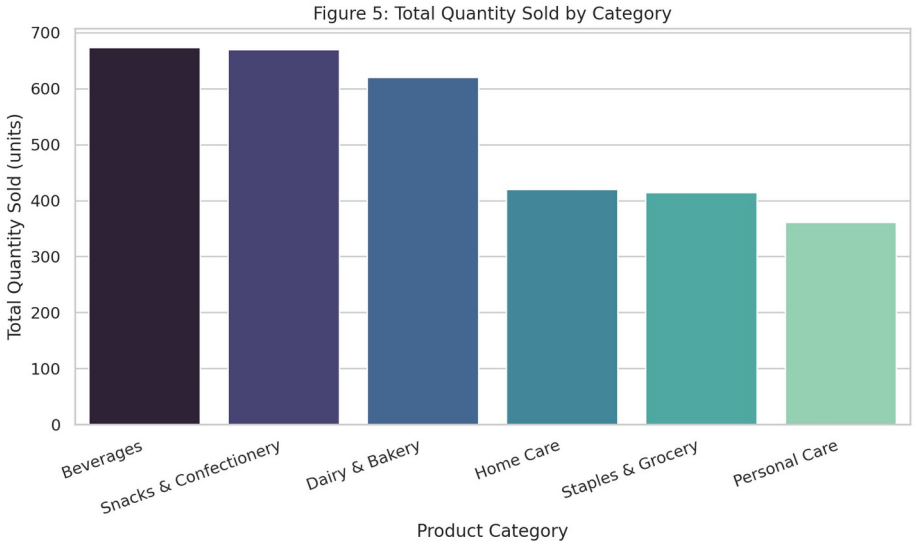


Figure 6: Top 10 Products by Final Sales

Command / Code

```
prod_sales = df.groupby("Product_Name")["Final_Sales"].sum().sort_values(ascending=False)
top10 = prod_sales.head(10).sort_values()
plt.figure(figsize=(9, 5.5))
plt.barh(top10.index, top10.values, color="teal")
plt.title("Figure 6: Top 10 Products by Final Sales")
plt.xlabel("Total Final Sales (Rs.)")
plt.ylabel("Product Name")
plt.tight_layout()
plt.show()
```

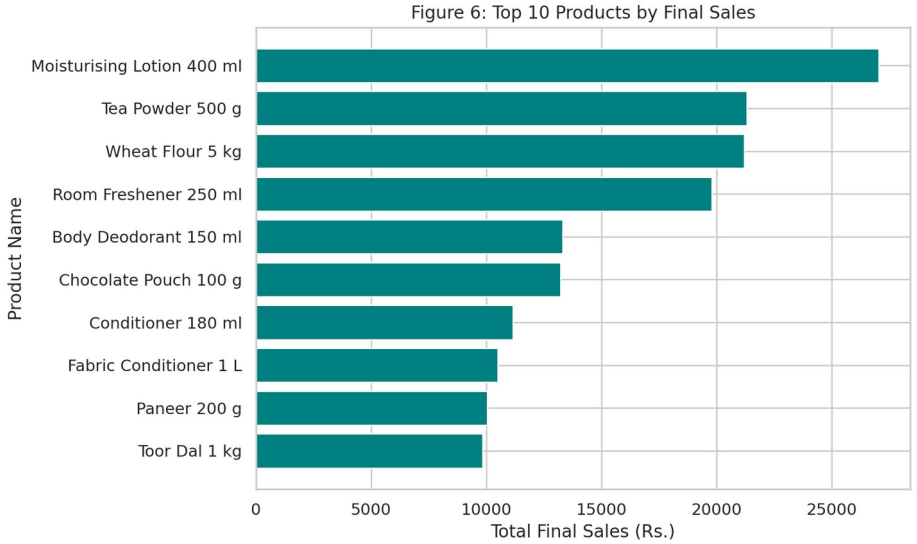


Figure 7: Bottom 10 Products by Final Sales

Command / Code

```
bottom10 = prod_sales.tail(10).sort_values(ascending=False)
plt.figure(figsize=(9, 5.5))
plt.barh(bottom10.index, bottom10.values, color="indianred")
plt.title("Figure 7: Bottom 10 Products by Final Sales")
plt.xlabel("Total Final Sales (Rs.)")
plt.ylabel("Product Name")
plt.tight_layout()
plt.show()
```

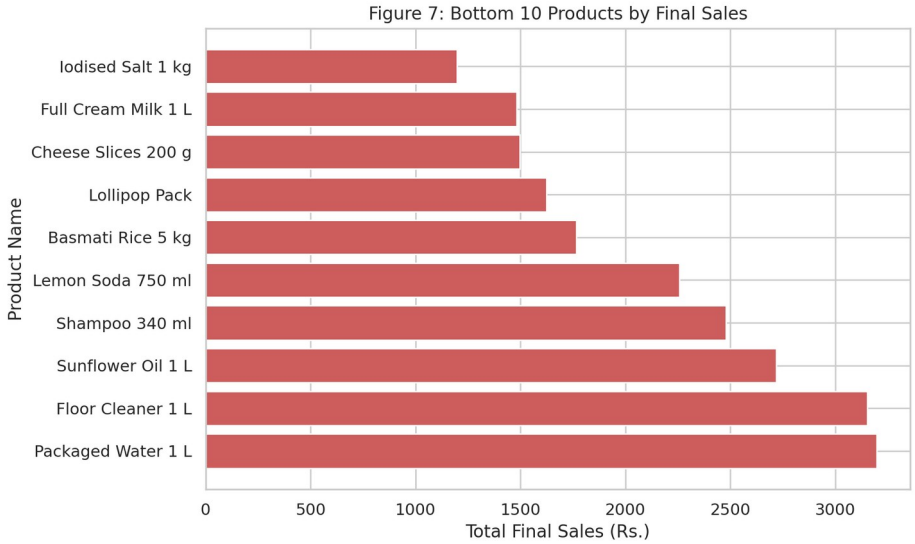


Figure 8: Share of Each Category in Total Final Sales (Donut Chart)

Command / Code

```
plt.figure(figsize=(9, 6))
plt.pie(cat_sales.values, autopct="%1.1f%%", startangle=90, pctdistance=0.78,
        wedgeprops={"width": 0.45, "edgecolor": "white"})
plt.legend(cat_sales.index, title="Category", loc="center left",
           bbox_to_anchor=(1, 0.5))
plt.title("Figure 8: Category Share in Total Final Sales")
plt.axis("equal")
plt.tight_layout()
plt.show()
```

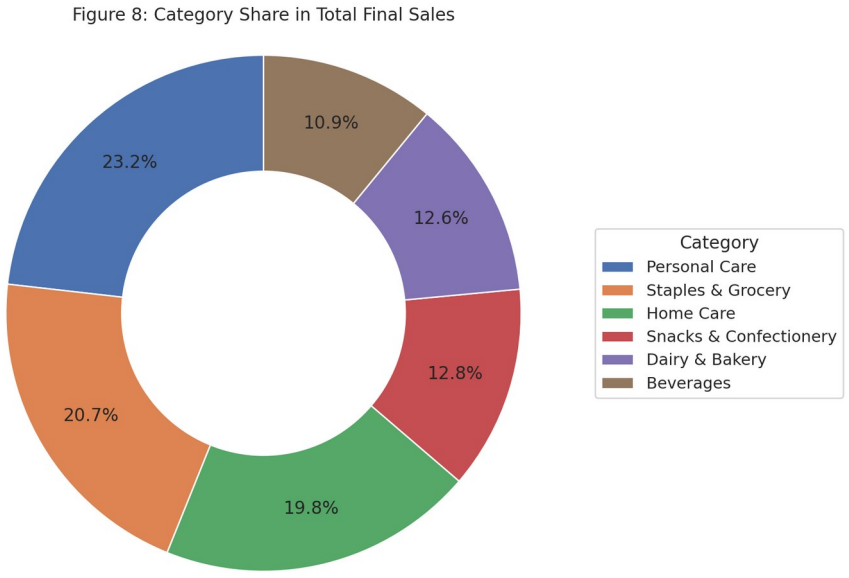


Figure 9: Distribution of Unit Price

Command / Code

```
plt.figure(figsize=(9, 5.5))
sns.histplot(df["Unit_Price"], bins=15, kde=True, color="steelblue")
plt.title("Figure 9: Distribution of Unit Price")
plt.xlabel("Unit Price (Rs.)")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
```

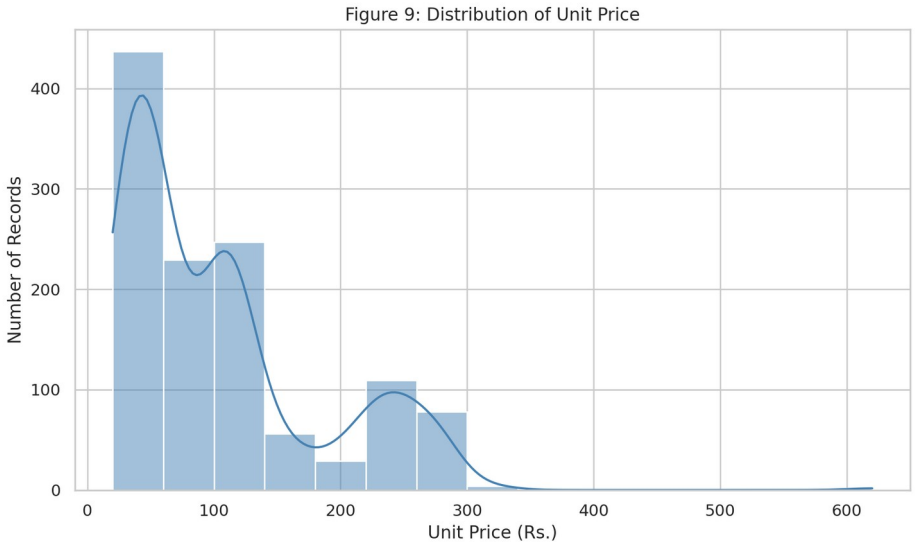


Figure 10: Distribution of Final Sales

Command / Code

```
plt.figure(figsize=(9, 5.5))
sns.histplot(df["Final_Sales"], bins=30, kde=True, color="darkorange")
plt.title("Figure 10: Distribution of Final Sales per Record")
plt.xlabel("Final Sales (Rs.)")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
```

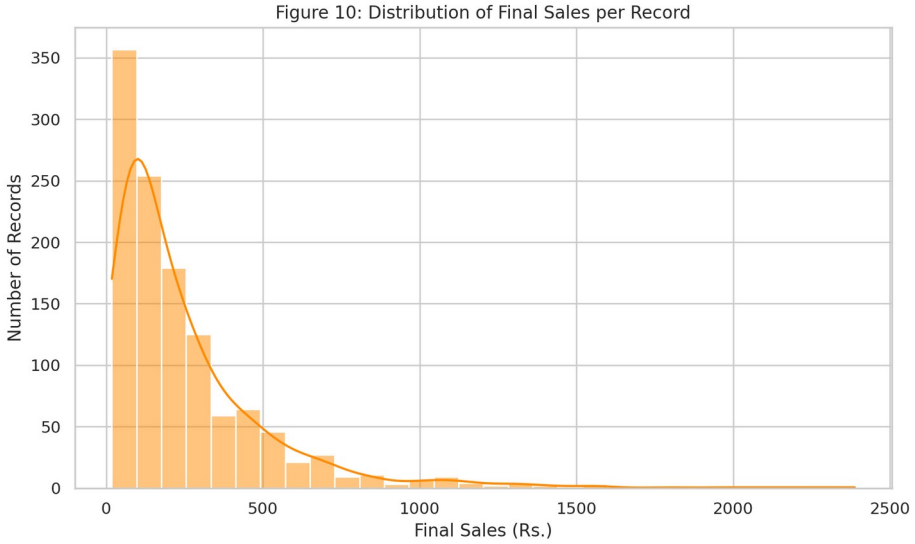


Figure 11: Boxplot of Final Sales

Command / Code

```
plt.figure(figsize=(9, 4.5))
sns.boxplot(x=df["Final_Sales"], color="lightgreen")
plt.title("Figure 11: Boxplot of Final Sales")
plt.xlabel("Final Sales (Rs.)")
plt.ylabel("All Records")
plt.tight_layout()
plt.show()
```

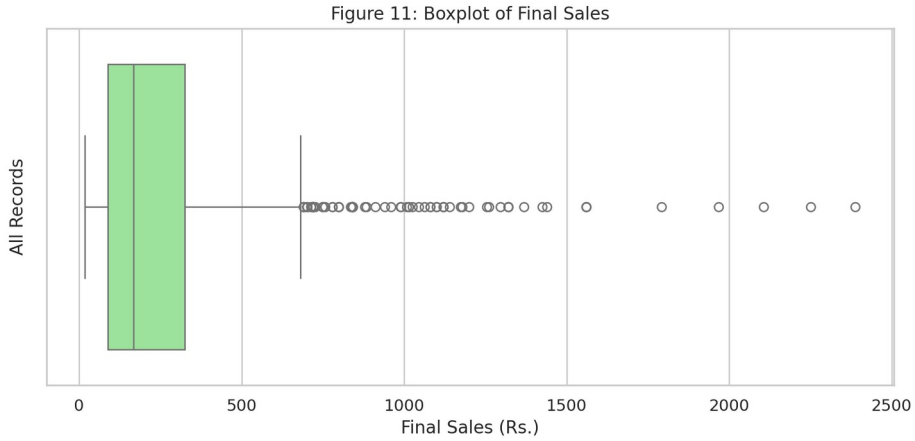


Figure 12: Boxplot of Unit Price by Category

Command / Code

```
plt.figure(figsize=(9, 5.5))
sns.boxplot(data=df, x="Category", y="Unit_Price", hue="Category",
            palette="Set2", legend=False)
plt.title("Figure 12: Unit Price by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Unit Price (Rs.)")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```

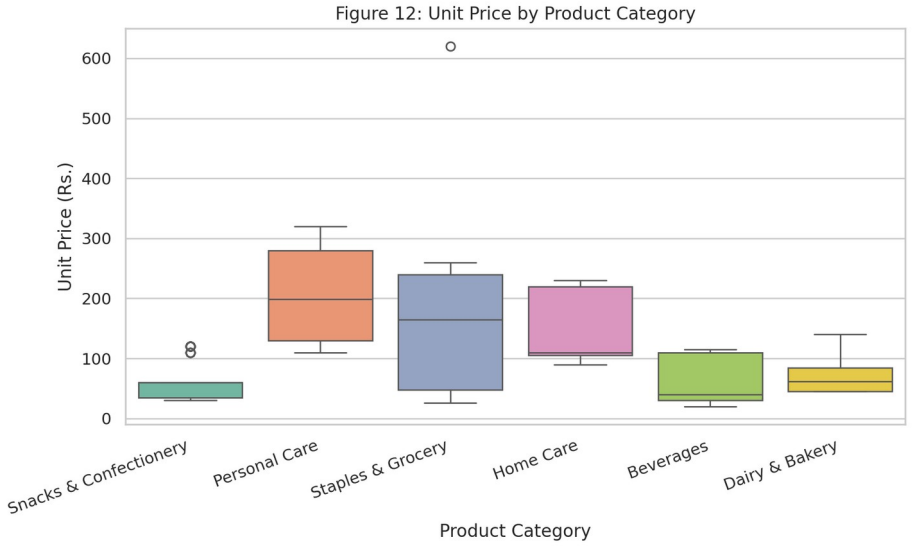


Figure 13: Daily Sales Trend with 7-Day Moving Average

Command / Code

```
daily = df.groupby("Date")["Final_Sales"].sum()
rolling = daily.rolling(7).mean()
plt.figure(figsize=(10, 5.5))
plt.plot(daily.index, daily.values, color="lightsteelblue", label="Daily Sales")
plt.plot(rolling.index, rolling.values, color="darkred", linewidth=2,
         label="7-Day Moving Average")
plt.title("Figure 13: Daily Sales Trend (July - September 2026)")
plt.xlabel("Date")
plt.ylabel("Final Sales (Rs.)")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

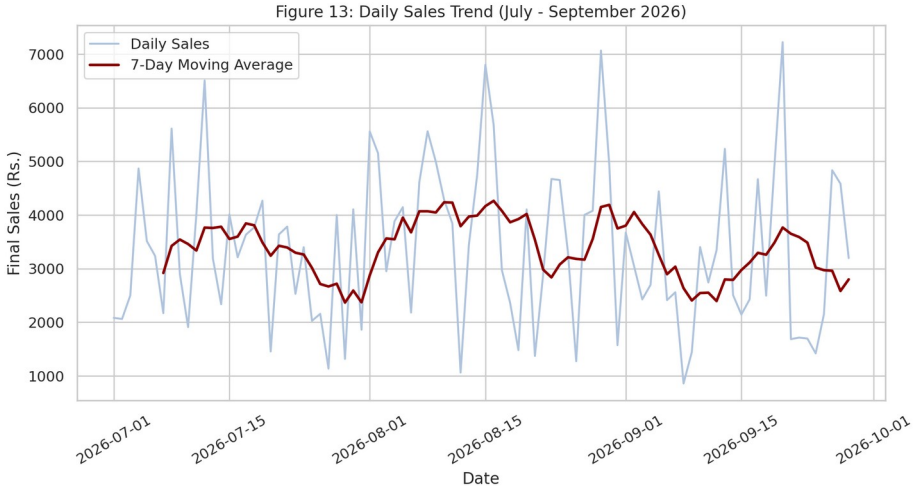


Figure 14: Monthly Sales Comparison

Command / Code

```
monthly = df.groupby(["Month_No", "Month"])["Final_Sales"].sum().reset_index()
plt.figure(figsize=(8, 5.5))
bars = plt.bar(monthly["Month"], monthly["Final_Sales"],
               color=["#4c72b0", "#55a868", "#c44e52"])
plt.bar_label(bars, fmt="%.0f")
plt.title("Figure 14: Monthly Sales Comparison")
plt.xlabel("Month (2026)")
plt.ylabel("Total Final Sales (Rs.)")
plt.tight_layout()
plt.show()
```

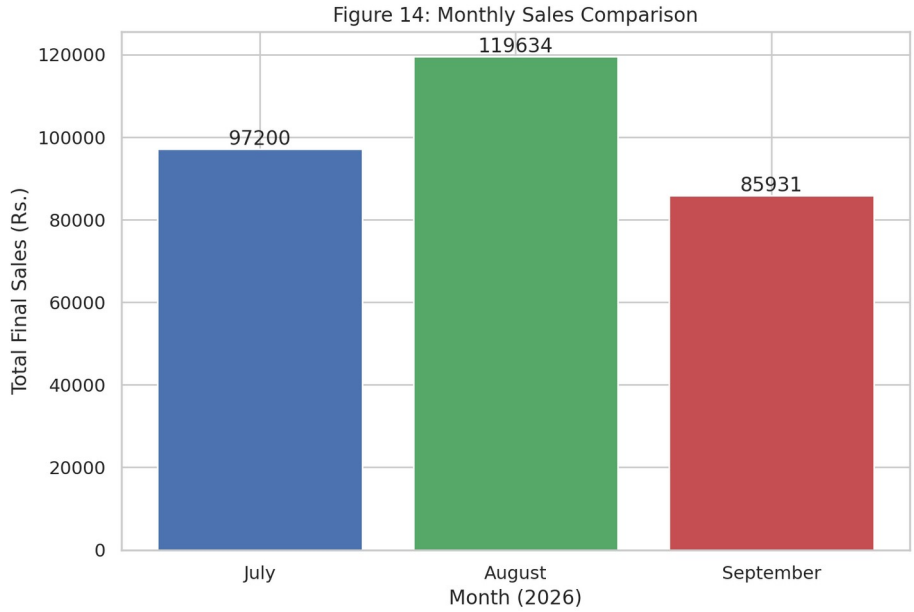


Figure 15: Discount Percentage versus Final Sales

Command / Code

```
plt.figure(figsize=(9, 5.5))
sns.scatterplot(data=df, x="Discount_Percent", y="Final_Sales",
                hue="Customer_Type", alpha=0.5)
plt.title("Figure 15: Discount Percentage vs Final Sales")
plt.xlabel("Discount (%)")
plt.ylabel("Final Sales (Rs.)")
plt.legend(title="Customer Type")
plt.tight_layout()
plt.show()
```

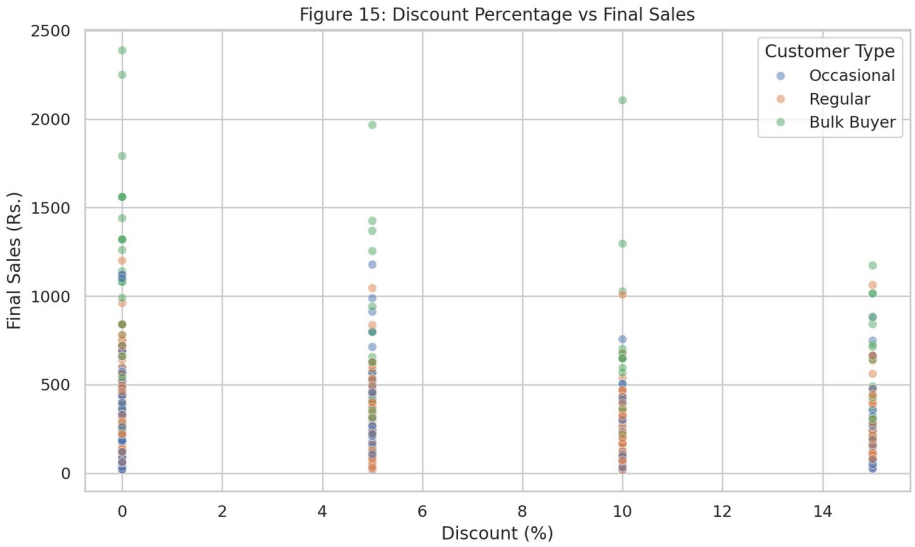


Figure 16: Quantity versus Total Sales

Command / Code

```
plt.figure(figsize=(9, 5.5))
sns.scatterplot(data=df, x="Quantity", y="Total_Sales",
                hue="Customer_Type", alpha=0.6)
plt.title("Figure 16: Quantity Sold vs Total Sales")
plt.xlabel("Quantity Sold (units)")
plt.ylabel("Total Sales (Rs.)")
plt.legend(title="Customer Type")
plt.tight_layout()
plt.show()
```

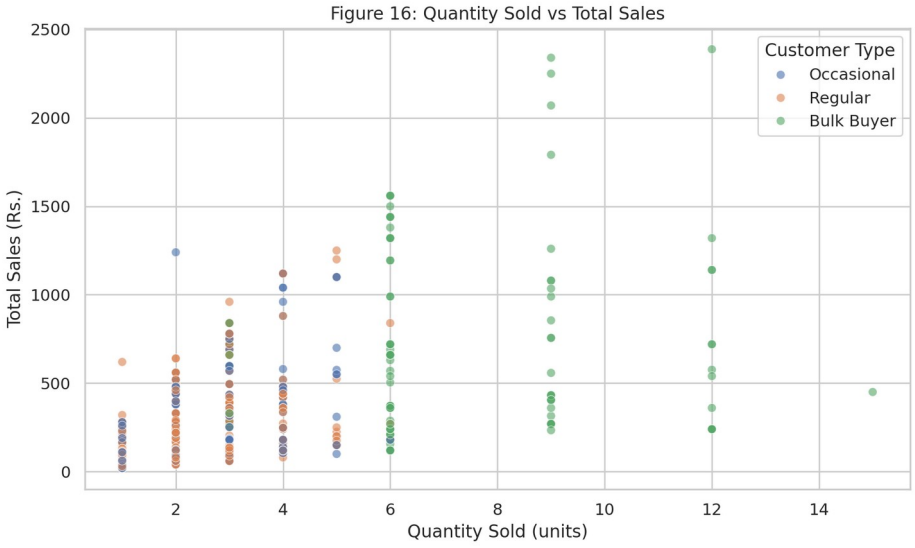


Figure 17: Payment Method Distribution

Command / Code

```
pay_counts = df["Payment_Method"].value_counts()
plt.figure(figsize=(8, 5.5))
bars = plt.bar(pay_counts.index, pay_counts.values, color="slateblue")
plt.bar_label(bars)
plt.title("Figure 17: Payment Method Distribution")
plt.xlabel("Payment Method")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
```

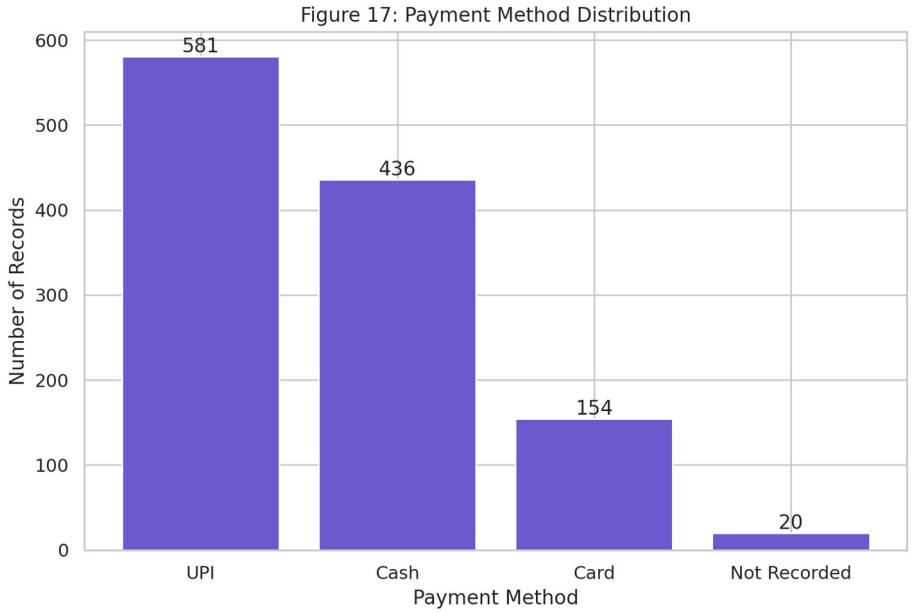


Figure 18: Customer Type Distribution

Command / Code

```
cust_counts = df["Customer_Type"].value_counts()
plt.figure(figsize=(8, 5.5))
bars = plt.bar(cust_counts.index, cust_counts.values, color="seagreen")
plt.bar_label(bars)
plt.title("Figure 18: Customer Type Distribution")
plt.xlabel("Customer Type")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
```

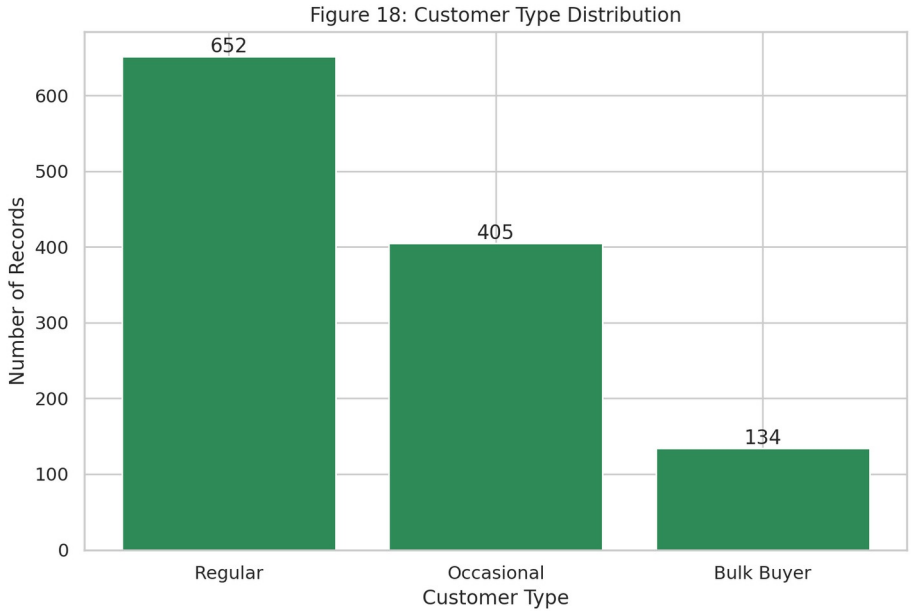
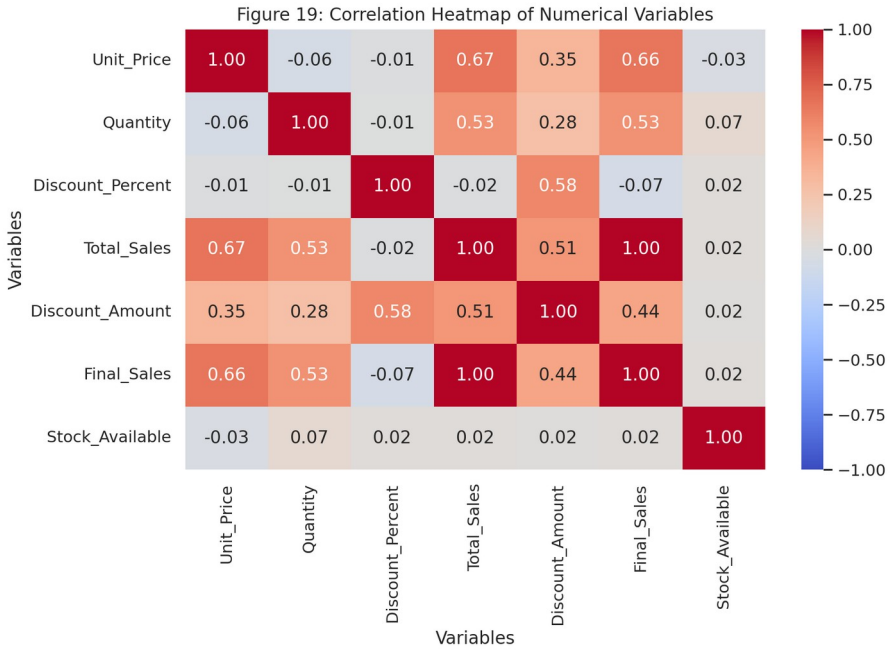


Figure 19: Correlation Heatmap of Numerical Variables

Command / Code

```
num = df[["Unit_Price", "Quantity", "Discount_Percent", "Total_Sales",
          "Discount_Amount", "Final_Sales", "Stock_Available"]]
plt.figure(figsize=(9, 6.5))
sns.heatmap(num.corr(), annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Figure 19: Correlation Heatmap of Numerical Variables")
plt.xlabel("Variables")
plt.ylabel("Variables")
plt.tight_layout()
plt.show()
```



15. ANALYSIS AND INTERPRETATION

Figure 4 (Total Final Sales by Product Category): The bar chart shows that Personal Care recorded the highest total final sales (Rs. 70,236.10), followed by Staples & Grocery (Rs. 62,671.25). Beverages recorded the lowest sales (Rs. 33,116.75) among the six categories.

Figure 5 (Total Quantity Sold by Category): In terms of units sold, Beverages led with 674 units and Personal Care was lowest with 361 units. The category with the highest sales value and the category with the highest quantity are different, which shows that unit price and quantity together decide the sales value.

Figure 6 (Top 10 Products by Final Sales): The top-selling product by final sales is Moisturising Lotion 400 ml (Rs. 27,048.00), followed by Tea Powder 500 g and Wheat Flour 5 kg. The top 10 products together contribute 52.0% of total final sales.

Figure 7 (Bottom 10 Products by Final Sales): The lowest final sales were recorded for Iodised Salt 1 kg (Rs. 1,197.30), followed by Full Cream Milk 1 L and Cheese Slices 200 g. Such products may need attention in pricing, display or stocking.

Figure 8 (Share of Each Category in Total Final Sales (Donut Chart)): The donut chart shows that Personal Care accounts for 23.2% of total final sales, while Beverages accounts for 10.9%. The remaining categories lie between these two values.

Figure 9 (Distribution of Unit Price): The histogram shows that most records belong to lower-priced products. The most common price band is Rs. 0-50 with 437 records. The mean unit price is Rs. 103.92 while the median is Rs. 84.00.

Figure 10 (Distribution of Final Sales): The distribution of final sales per record is right-skewed. Most records fall at lower values, while a few high-value records extend the right tail. The mean (Rs. 254.21) is higher than the median (Rs. 168.00).

Figure 11 (Boxplot of Final Sales): The boxplot shows a median final sale of Rs. 168.00 with the middle 50% of records between Rs. 89.25 and Rs. 325.88. There are 76 records above the upper whisker, which represent high-value purchases such as bulk buying or costly items.

Figure 12 (Boxplot of Unit Price by Category): The boxplot shows that Personal Care has the highest median unit price (Rs. 199.00), while Beverages has the lowest (Rs. 40.00). Categories with wider boxes contain products with a wider range of prices.

Figure 13 (Daily Sales Trend with 7-Day Moving Average): The daily sales line fluctuates from day to day, and the 7-day moving average smooths these changes. The highest daily sales were recorded on 20-09-2026 (Rs. 7,222.40). The average sales per weekend day (Rs. 4,670.49) were higher than per weekday (Rs. 2,833.32).

Figure 14 (Monthly Sales Comparison): Total final sales were Rs. 97,200.30 in July, Rs. 119,633.75 in August and Rs. 85,930.85 in September. August recorded the highest sales. Note that the September data ends on 28-09-2026, so it does not cover the full month.

Figure 15 (Discount Percentage versus Final Sales): The scatter plot shows almost no relationship between discount percentage and final sales (correlation = -0.07). High-value

records appear at all discount levels, so discount alone does not explain the sales value in this dataset.

Figure 16 (Quantity versus Total Sales): The scatter plot shows a moderate positive relationship between quantity sold and total sales (correlation = 0.53). Points are spread in separate lines because each line represents products of a similar unit price.

Figure 17 (Payment Method Distribution): UPI was the most used payment method with 581 records (48.8%), followed by Cash (36.6%). Card accounted for 12.9%.

Figure 18 (Customer Type Distribution): Regular customers formed the largest group (54.7% of records), followed by Occasional customers (34.0%) and Bulk Buyer customers (11.3%).

Figure 19 (Correlation Heatmap of Numerical Variables): As expected, Total_Sales, Discount_Amount and Final_Sales are strongly related to each other because they are calculated from the same variables. Among the other variables, the strongest relationship is between Unit_Price and Total_Sales (correlation = 0.67). Stock_Available shows very little relationship with the sales variables.

Interpretation of Important Tables

Category-wise summary: Personal Care had the highest final sales (Rs. 70,236.10), while Beverages had the lowest (Rs. 33,116.75). Discounts reduced total sales from Rs. 317,577.00 to Rs. 302,764.90.

Brand-wise summary: CreamyCow was the leading brand by final sales (Rs. 31,435.65), followed by SoftSkin (Rs. 27,048.00).

Customer-type summary: Bulk Buyer customers had the highest average final sales per record (Rs. 571.24), and Bulk Buyer customers bought the highest average quantity (6.27 units per record).

16. KEY FINDINGS

- The cleaned dataset contains 1,191 records and 23 columns (including the columns created during processing), covering 42 products in 6 categories.
- Total quantity sold was 3,160 units.
- Total sales before discount were Rs. 317,577.00; discounts amounted to Rs. 14,812.10 (4.7%), giving total final sales of Rs. 302,764.90.
- The average final sales per record was Rs. 254.21; the minimum was Rs. 18.00 and the maximum was Rs. 2,388.00.
- The highest-selling category by value was Personal Care (23.2% of final sales); the lowest was Beverages (10.9%).
- The most frequently occurring category in the records was Snacks & Confectionery (259 records).
- The highest-selling product by final sales was Moisturising Lotion 400 ml (Rs. 27,048.00); the most units were sold of Flavoured Milk 200 ml (174 units).
- The lowest-selling product by final sales was Iodised Salt 1 kg (Rs. 1,197.30).
- The leading brand by final sales was CreamyCow (Rs. 31,435.65).

- The most common price band was Rs. 0-50; the median unit price was Rs. 84.00.
- The most common discount level was 0%. The correlation between discount percentage and final sales was -0.07.
- Monthly final sales: July Rs. 97,200.30, August Rs. 119,633.75, September (till 28th) Rs. 85,930.85.
- Average sales per weekend day were Rs. 4,670.49 compared with Rs. 2,833.32 per weekday.
- UPI was the most used payment method (48.8% of records).
- Regular customers formed 54.7% of records; Bulk Buyer customers had the highest average final sales per record (Rs. 571.24).

17. CONCLUSION

In this field project, a supermarket was visited and the type of data generated at a supermarket was studied through geotagged photographs. A simulated sales dataset of 1,215 records was used for analysis, and it is clearly stated that this dataset does not represent the actual records of the visited shop. Using Python, the raw data was cleaned by handling missing values, removing duplicate and invalid records and correcting category names and data types, which gave 1,191 valid records.

After processing, the analysis showed total final sales of Rs. 302,764.90 from 3,160 units sold. Personal Care was the highest-selling category and Beverages the lowest, and Moisturising Lotion 400 ml was the highest-selling product. Most records belonged to the price band Rs. 0-50. UPI was the most used payment method and Regular customers formed the largest customer group. The graphs made using Matplotlib and Seaborn presented these results clearly.

The project shows that Python, with pandas, NumPy, Matplotlib and Seaborn, makes it easy to clean, summarise and visualise supermarket data. It also showed how the records created at a billing counter can be converted into useful information about products, categories, pricing and customer purchases.

18. LIMITATIONS

- The field observation was limited to a single short visit.
- Confidential sales records of the supermarket were not accessible.
- The dataset is simulated; hence the results describe the sample data and not the actual shop.
- The dataset is small (1,191 valid records for about three months).
- Customer information is limited to customer type and payment method.
- Cost price data is not available, so profit could not be analysed.
- The analysis depends only on the variables available in the dataset.

19. FUTURE SCOPE

- Analysis of real, permitted sales data of a supermarket.
- Real-time sales analytics.
- Inventory prediction to avoid overstocking and stock-outs.
- Customer segmentation using purchase behaviour.
- Demand forecasting for busy days and festival seasons.
- Recommendation systems for products often bought together.
- Sales prediction using machine learning models.
- Automated dashboards for the shop owner.

20. REFERENCES

1. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

2. The pandas development team. pandas Documentation. https://pandas.pydata.org/docs/
3. NumPy Developers. NumPy Documentation. https://numpy.org/doc/
4. The Matplotlib development team. Matplotlib Documentation. https://matplotlib.org/stable/
5. Waskom, M. seaborn: statistical data visualization (Documentation). https://seaborn.pydata.org/
6. Google Colaboratory. https://colab.research.google.com/
7. McKinney, W. Python for Data Analysis, 3rd Edition. O'Reilly Media, 2022.
8. Dataset: Simulated supermarket sales dataset created by the student using Python (code given in Section 2), file supermarket_sales_raw.csv.
9. Field visit photographs: taken by the student on 29/09/2026 using the GPS Map Camera application.